

Práctica 1 de CESI

Joaquín Cuéllar Padilla joaquin.cuellar@uco.es

Septiembre de 2025

Resumen

En esta práctica vamos a dar un recorrido por el hardware que comúnmente compone o puede componer un sistema informático. Es una práctica enfocada a la cultura general que debería poseer toda persona que aspire a obtener una Ingeniería Informática, en lo referente a componentes informáticos. Los objetivos son:

- Identificar los componentes básicos y comunes que podemos encontrar en un equipo informático.
- Identificar los componentes que componen nuestro propio equipo informático, saber qué empleamos como herramienta de trabajo diaria.
- Ser consciente de los componentes y equipos disponibles en el mercado.
- Tener recursos y conocimientos básicos para reparar u obtener un nuevo equipo informático.

1. Descripción de la práctica

1.1. Objetivo de la práctica

Esta práctica tiene un enfoque muy de Formación Profesional y práctico.

El objetivo es que tengamos soltura y recursos a la hora de saber identificar los componentes de nuestro equipo informático como herramienta de trabajo.

Este conocimiento es básico en una ingeniería informática, pues el Hardware estará estrechamente ligado con las capacidades de cómputo de nuestro equipo, sistemas operativos instalables, software libre y privacidad, etc.

1.2. Organización de la práctica

Va a estar dividida en **dos sesiones**

1.3. ¿Qué se os va a pedir?

Solamente vuestra atención y participación activa!

Las prácticas **no son de asistencia obligatoria**

Pero sí **entrarán en un examen de prácticas a final de curso**

Aún así creemos que son interesantes y atractivas, pues vamos a poner a vuestra disposición información y recursos para saber 'cacharrear' hardware y administrar sistemas informáticos GNU/Linux.

2. Qué vamos a realizar.

2.1. 1ª Sesión

En esta sesión vamos a identificar cada uno de los componentes de nuestro equipo informático

En el aula disponemos de los antiguos equipos, despiezados, que aún siendo antiguos, nos van a permitir introducir la materia, darnos componentes que poder identificar y manipular y ofrecernos un punto de partida para presentar las tecnologías actualmente existentes en la industria y el mercado.

En kit de trabajo lo conforman:

2.1.1. La placa base / Motherboard



Figura 1: Detalle de la placa base presente en el aula.



Figura 2: Puertos de conectividad que nos ofrece la placa base presente en el aula.

¿Qué hace este componente?

Nos va a permitir interconectar todos los componentes del equipo y tendrá compatibilidad con un hardware determinado

De él va a depender la Bios, y el posible hardware extra asociado: red, GPU, puertos disponibles, discos duros, periféricos... (ya cada vez más hardware viene dentro de los procesadores, pero este se tiene que seguir conectando a una placa base.)

En el caso concreto de la placa base del aula, es interesante comprobar que nos ofrece compatibilidad con los módulos de RAM y Procesador existente (podéis ver en su documentación oficial que con más tipos).

Ofrece una tarjeta gráfica ya incluida, y conexión de red. A parte permite la conexión de discos duros y periféricos tanto PATA como SATA, se explican más adelante.

¿Como la identifico?

La clave está en encontrar el fabricante (Gigabyte) y su modelo, en este caso es **GA-945GCMX-S2**

Con esta referencia podemos acudir a la web del fabricante (si ofrece buena documentación, claro!): [especificaciones GigaByte GA-945GCMX-S2](#).



Figura 3: Detalle del modelo la placa base presente en el aula.

¿Qué debemos valorar de una placa base?

- Compatibilidad con el resto de componentes, qué procesadores puede emplear, qué RAM, etc.
- ¿Qué componentes trae ya integrados? Wifi? GPU? Ethernet?
- ¿Como es de ampliable y modificable? ¿Puedo cambiar la CPU o viene soldada? ¿Puedo cambiar la memoria RAM o también viene soldada? ¿Qué puertos de expansión tiene (PCI express, etc.)?
- Precio!

Referencias de lo comentado

- [Definición de la Wikipedia sobre Placa Base.](#)
- [Definición de BIOS](#)
- [Libreboot, una BIOS de software libre.](#)

2.1.2. El procesador /CPU

¿Qué hace este componente?

(de la wikipedia) El procesador o microprocesador es la unidad de procesamiento principal de un ordenador, es por ello la unidad más importante, el «cerebro» de un ordenador.

Es el encargado de ejecutar todos los programas, desde el sistema operativo hasta las aplicaciones de usuario; solo ejecuta instrucciones en lenguaje máquina, realizando operaciones aritméticas y lógicas simples, tales como sumar, restar, multiplicar, dividir y las operaciones bitwise, también las lógicas binarias y accesos a memoria.

Es importante que lo identifiquemos, pues de su arquitectura y capacidades nos darán noción de qué opciones tenemos a la hora de poder optimizar algoritmos y las capacidades de cómputo.

¿Como la identifico?

En nuestro caso, como lo tenemos físicamente en las prácticas, podemos identificarlo mediante el código mismo serigrafiado en el componente. **Intel Pentium Dual Core E2140**

En los casos en los que eso no sea posible, podemos identificarlo mediante la documentación oficial del fabricante de nuestro equipo informático. En el caso de ordenadores portátiles, por ejemplo.



Figura 4: Detalle del procesador presente en el aula.

Otra opción es emplear las utilidades de nuestro sistema operativo, es sencillo probar una distribución GNU/Linux que pueda iniciar desde el usb en un equipo, y una vez dentro del sistema analizar el procesador con las herramientas que vienen de serie.

```
joa@joa-EXECUTIVE-14: ~  
joa@joa-EXECUTIVE-14:~$ lscpu  
Arquitectura: x86_64  
modo(s) de operación de las CPUs: 32-bit, 64-bit  
Address sizes: 39 bits physical, 48 bits virtual  
Orden de los bytes: Little Endian  
CPU(s): 8  
Lista de la(s) CPU(s) en línea: 0-7  
ID de fabricante: GenuineIntel  
Nombre del modelo: Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz  
Familia de CPU: 6  
Modelo: 126  
Hilo(s) de procesamiento por núcleo: 2  
Núcleo(s) por «socket»: 4  
«Socket(s)»: 1  
Revisión: 5  
CPU MHz máx.: 3900,0000  
CPU MHz mIn.: 400,0000  
BogoMIPS: 2995.20  
Indicadores: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush  
_perfmon pebs bts rep good nopl xtopology nonstop_tsc cpuid aperfmperf tsc_knd  
sse4_1 sse4_2 x2apic movbe popcnt tsc deadline_timer aes xsave avx f16c rdrand  
tpr_shadow vnmi flexpriority ept vpid ept_ad fsgsbase tsc_adjust sgx bmi1 avx2  
12cd sha_ni avx512bw avx512vl xsaveopt xsavec xgetbv1 xsave split_lock detect  
ku ospke avx512_vbmi2 gfni vaes vpclmulqdq avx512_vnni avx512_bitalg avx512_vp  
Virtualization features:  
Virtualización: VT-x
```

Figura 5: Salida del comando por terminal 'lscpu'.

```
joa@joa-EXECUTIVE-14:~$ cat /proc/cpuinfo  
processor : 0  
vendor_id : GenuineIntel  
cpu family : 6  
model : 126  
model name : Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz  
stepping : 5  
microcode : 0xca  
cpu MHz : 1144.993  
cache size : 8192 KB  
physical id : 0  
siblings : 8  
core id : 0  
cpu cores : 4  
apicid : 0  
initial apicid : 0  
fpu : yes  
fpu_exception : yes  
cpuid level : 27  
wp : yes  
flags : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse  
ood nopl xtopology nonstop_tsc cpuid aperfmperf tsc_known_freq pni pclmulqdq dtes64 monitor ds_cpl vmx est tm2 ssse3 sc  
x f16c rdrand lahf_lm abm 3dnowprefetch cpuid_fault epb invpcid_single ssbd tbrs_lbrs stibp tbrs_enhanced tpr_shadow vr  
512f avx512dq rdseed adx snap avx512ifma clflushopt intel_pt avx512cd sha_ni avx512bw avx512vl xsaveopt xsavec xgetbv1  
kgf req avx512vbmi umip pku ospke avx512_vbmi2 gfni vaes vpclmulqdq avx512_vnni avx512_bitalg avx512_vpopcntdq rdpid sgx  
vmx flags : vnmi preemption_timer posted_intr invvpid ept_x_only ept_ad ept_1gb flexpriority apicv tsc_offset vtp
```

Figura 6: Salida del comando por terminal 'cat /proc/cpuinfo'.

¿Qué debemos valorar de un procesador?

- Arquitectura del procesador. (x86_64, ARM64, Risc-V, MIPS...)
- ¿Qué componentes trae ya integrados? GPU? Wifi? Bluetooth? [NPU](#)?...
- Número y tipo de núcleos, velocidad de cada núcleo... (threads?)
- [Tecnología de fabricación](#)
- Cantidad de Memoria Caché, y de qué nivel (L1, L2, L3).
- Consumo eléctrico en W. (Watts)
- Precio!

Referencias de lo comentado

- [Definición de Procesador.](#)
- [Definición de arquitectura de un procesador](#)
- [Ejemplo de listado de procesadores Intel, para ver sus especificaciones.](#)
- [Ejemplo de listado de procesadores AMD Ryzen, para ver sus especificaciones.](#)
- [Ejemplo de listado de procesadores Loongson \(Arquitectura MIPS\), para ver sus especificaciones.](#)

2.1.3. La memoria RAM

¿Qué hace este componente?

[\(de la wikipedia\)](#) La memoria de acceso aleatorio (Random Access Memory, RAM) es una memoria de almacenaje a corto plazo. El sistema operativo de ordenadores u otros dispositivos utiliza la memoria RAM para guardar de forma temporal todos los programas y sus procesos de ejecución.

En la RAM se cargan todas las instrucciones que ejecuta la unidad central de procesamiento (CPU) y otras unidades del ordenador, además de contener los datos que manipulan los distintos programas...

¿Como la identifico?

En nuestro caso, como también lo tenemos físicamente en las prácticas, podemos identificarlo mediante el código mismo en una pegatina sobre el componente. **Kingston KVR667D2N5/1G**

¿Qué debemos valorar de una memoria RAM?



Figura 7: Detalle de uno de los módulos de RAM mostrados en el aula.

- Formato de la RAM: DIMM, SO-DIMM...
- Tipo de RAM, DDR4, DDR5, LPDDR... Ojo con las RAMS soldadas.
- Velocidad de transmisión de la RAM en Mhz.
- [Latencia de la memoria](#) (menor latencia mejor)
- Empleo o posibilidad de dos módulos idénticos de memoria para [dual channel](#) Dependerá de la placa base/CPU y aumenta mucho el rendimiento.
- Precio!

Referencias de lo comentado

- [Definición de Ram.](#)
- [infiernillo documentado para ampliar una RAM soldada.](#)



Figura 8: Detalle de uno de los discos duros de 3.5" ATA presentes en el aula.

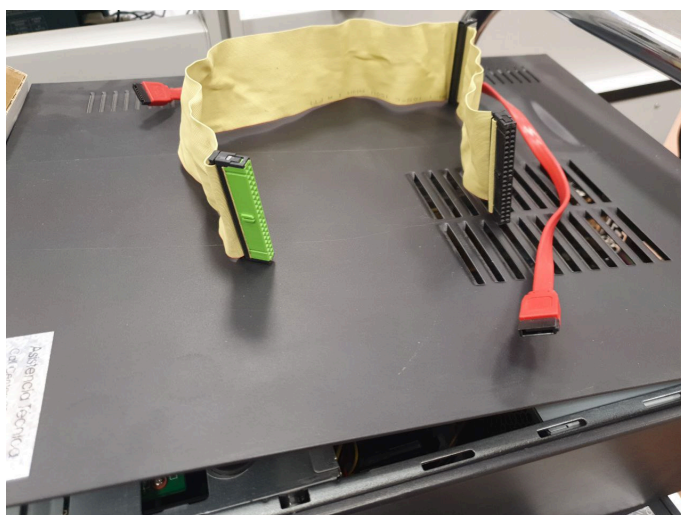


Figura 9: Detalle de cables ATA (PATA) y SATA.



Figura 10: Detalle discos duros de 2.5" SATA uno mecánico y otro SSD.

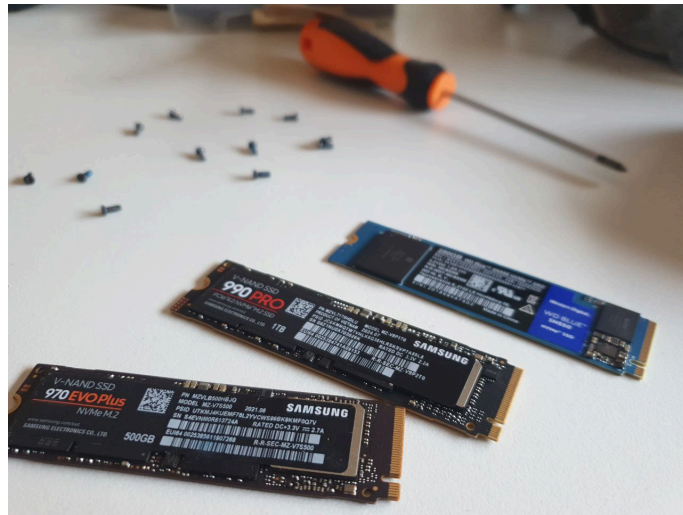


Figura 11: Detalle de discos duros SSD más actuales de tipo nvme.

2.1.4. Disco duro (memoria persistente)

¿Qué hace este componente?

(de la wikipedia) El disco duro es el componente de nuestro equipo informático que se encargará de almacenar la información que se quiera mantener persistente. Datos, programas, etc.

El disco duro puede hacer de 'cuello de botella' en los sistemas si se necesita mucho acceso a datos del sistema, así que no es un elemento despreciable en cuanto a prestaciones (**y configuración! En la siguiente práctica veremos la importancia del particionado del disco duro**). Pensad que incluso algunos sistemas operativos emplean como SWAP o memoria de intercambio cuando falta memoria RAM, el disco duro.

¿Como lo identifico?

Es un componente de tamaño considerable y normalmente desenchufable de portátiles y equipos de sobremesa, gracias a eso su información viene accesible.

¿Qué debemos valorar de un disco duro

- ¿Cuántos componentes de este tipo podemos insertar en nuestra placa base? ¿Cuántos puertos PCI-e tenemos? o puertos SATA, etc.? Este es un factor limitante siempre.
- Planificar el particionado, queremos hacer dual boot? nos puede facilitar la vida dos discos duros.
- Capacidad del disco duro, un GNU/Linux Ubuntu actual puede verse muy limitado con 250 /500 Gb. A más alta, más datos podemos almacenar.
- La velocidad de transferencia de los datos es otro valor a tener en cuenta, a más velocidad mejor.
- Si tienes posibilidad de saber los ciclos (lectura y escritura) de vida útil del dispositivo, mejor! A Cuantos más ciclos más vida útil.

- Precio!

Referencias de lo comentado

- [Definición de Disco duro.](#)
- [IDE / ATA.](#)
- [SATA protocolo serie Half-Duplex.](#)
- [SAS protocolo serie Full-Duplex \(muy empleado en servidores\).](#)
- [Unidad de estado sólido](#)
- [Discos NVMe.](#)

2.2. 2ª Sesión

En esta sesión vamos a ver los retos que encontramos a la hora de poder emplear nuestros equipos con garantías de privacidad y de soporte de software libre, así como las opciones tenemos a la hora de adquirir equipos informáticos, respecto a su soporte con software libre.

Vamos a ver donde podemos buscar información sobre nuestros componentes y a las malas como probar nuestros equipos con software libre.

Finalmente voy a introducir equipos variados que os podréis encontrar en vuestra carrera friki.

2.2.1. Problemas! (retos)

Os vais a encontrar básicamente 4 tipos de problemas:

- El equipo no está soportado (aún)
- El equipo carece de algún driver, o lo necesitas de software cerrado.
- El equipo requiere de firmware cerrado.
- El equipo viene mal diseñado o no es flexible a ser modificado.

En el caso de que el equipo no esté soportado (aún)

Esta situación es bastante común. Hay características que pueden no estar aún implementadas en el sistema operativo que elijamos, o que lo están a partir de cierta versión. Imaginaos que adquirís un nuevo equipo que posee un [Intel n250](#). Este procesador va a requerir que trabajéis con una versión del núcleo Linux igual o superior a la 6.11. [ref1](#), [ref2](#)

En este caso deberemos ajustar la versión del sistema operativo a una que traiga por defecto un kernel con esa versión o superior, por ejemplo: [Podremos emplear Ubuntu 25.04](#) , pero no [Debian 12, que la versión es la 6.1.](#)

Tema drivers y firmware

Muchos de los componentes del equipo necesitarán de módulos especiales, llamados drivers, en el sistema operativo, para ser controlados.

GNU/Linux es de sobra, el sistema operativo con mejor soporte para componentes, principalmente a su filosofía de retrocompatibilidad. En este caso **se deberá buscar por el componente en sí el estado de soporte para un sistema operativo concreto**

Los dispositivos más complejos suelen ser las tarjetas gráficas y las comunicaciones. (Wifi, Bluetooth...)

Respecto al [firmware o Blobs binarios](#) el tema es más complicado aún, muchas veces las empresas venden la posibilidad de emplear drivers de software libre, pero que requieren de micro-código a bajo nivel, compilado y de código cerrado para funcionar correctamente.

Las tarjetas gráficas y las comunicaciones suelen ser las más dependientes de estos blobs privativos. También los procesadores se suelen ver afectados. [Por ejemplo Intel](#), actualiza la funcionalidad de sus procesadores mediante correcciones software en forma de estos blobs privativos.

[Las bios](#) entrarían también en este campo del código cerrado que nuestro equipo necesita ejecutar y no sabemos como está hecho.

Ejemplos de proyectos importantes que intentan liberar la bios son: [GNU Boot](#), [Libreboot](#) y uno más permisivo pues acepta blobs privativos [Coreboot](#)

Por último nos podemos encontrar con equipos mal diseñados o cerrados por defecto.

Juntando mal diseño y firmware cerrado, podemos encontrarnos ejemplos de posibles malas prácticas y agujeros de seguridad con el famoso [Intel Management Engine](#) (AMD tiene el suyo también) y os comparto en temas de privacidad y seguridad como tener software cerrado que no controlamos puede ser un problema y como [empresas como google están evitando cualquier firmware cerrado en sus servidores.](#)

[Un ejemplo es el Raspberry PI, cuyo procesador debe de ser inicializado por la GPU y emplea un firmware cerrado con poca solución.](#)

[Otro ejemplo es en telefonía, donde tenemos procesadores potentes, con capacidad para ejecutar firmware cerrado interno, y con conectividad dentro del mismo SOC con todos los periféricos.](#)

2.2.2. Buscando info sobre soporte

H-node, proyecto de la FSF (os invito a participar!) <https://h-node.org>

En temas informáticos la wikipedia está muy completa. https://en.wikipedia.org/wiki/Comparison_of_open-source_wireless_drivers

Foros, chatgpt...

Hay vendedores que ofrecen equipos con GNU/Linux, e incluso con GNU/Linux-libre

- Slimbook, España, equipos con GNU/Linux. (ensamblados por ellos)
- Vant, España, equipos con GNU/Linux.
- Purism, equipos con GNU/Linux.
- System76, equipos con GNU/Linux.
- Tehnoetic, Rumanía, equipos con GNU/LinuxLibre!.

2.2.3. Si tenéis acceso al equipo, probad!

Distribuciones GNU/Linux con versión Live, para probarlar desde un USB

- Ubuntu
- Debian
- Fedora

Distribuciones GNU/Linux-Libre

- Trisquel
- Mas distribuciones...

2.2.4. Popurri de cacharros

Lemote Yeloong



Figura 12: Lemote Yeloong.

- Bios libre, proyecto pmon.

- Arquitectura MIPS64
- Mal diseño, la tarjeta gráfica no funciona bien con drivers de software libre.

Raspberry Pi

- [SBC](#)
- Arquitectura con mala fama, arranca desde la GPU y con firmware cerrado. Proyecto para liberar el firmware del raspberry pi. <https://github.com/librerpi/rpi-open-firmware>
- ARM 32 bits ARMv7

Odroid-C2

- ARM 64 bits ARMv8
- A parte de en una SD, el almacenaje lo puede hacer en una memoria Flash de tipo eMMC.

Raspberry pi pico

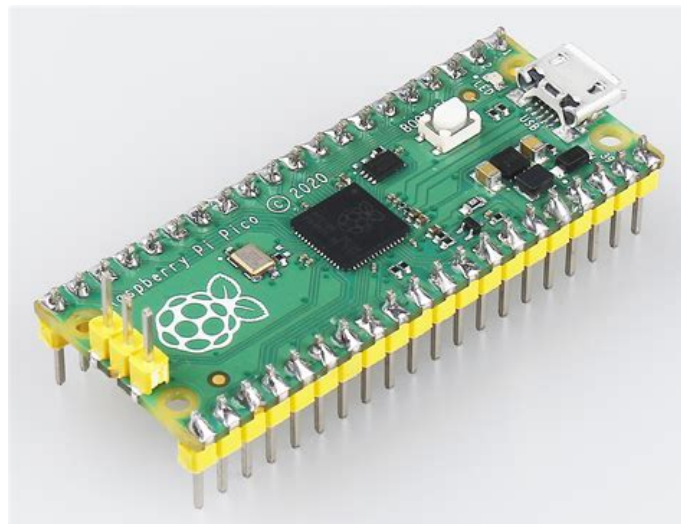


Figura 13: Raspberry Pi pico.

- 133 MHz dual ARM Cortex-M0+ cores (ARMv6-M)
- sin caché!
- sin MMU! No es una CPU es un [MCU](#)
- 264 KB SRAM

Esp32-c6

- Risc-V32 bits.
- Es un MCU.



Figura 14: ESP32-c6.

- HP core: 160Mhz, 4 stages pipeline, 512KB RAM.
- LP core: 20Mhz, 2 stages pipeline, 16 KB RAM.
- Arquitectura heterogénea!